Dataset Forensics and Label Audit

Dataset Forensics and Label Audit

Topic B — Final Project Report

Dataset: UCI SMS Spam Collection, 5,574 English text messages, split by the fixed course manifest into 4,460 training rows and 1,114 held-out rows **Task**: audit the quality of this dataset, locate suspicious examples across six issue types, and assess how data quality changes what the evaluation numbers mean

---

1. Summary

This report audits a spam-classification dataset built from the public UCI SMS corpus. It covers the six audit targets set by the handout and compares nine data interventions. Four findings stand out.

First, the 99.10% held-out accuracy overstates how hard the task is. 63 held-out rows (5.7% of the split) can be recovered from the training set, and the reference baseline gets every one of them right. Removing them from scoring drops spam recall from 93.96% to 90.91%.

Second, shallow surface features explain most of the performance. A logistic regression probe that receives only hand-built contact and promotional features, and never sees word order, reaches 98.11% accuracy and 92.78% spam F1. The full word model reaches 99.10% and 96.55%. The gap is about one percentage point. The dataset contains a strong shortcut: a model can score near the ceiling without representing sentence meaning.

Third, annotation convention is now the main source of noise in this benchmark. Of the 10 errors the baseline makes on held-out data, 7 are genuine model errors and 3 are data problems (2 suspected label errors and 1 ambiguous policy case). With the data as it stands, no model can exceed 99.73% held-out accuracy. The entire remaining headroom above the current score is 7 rows, which is the same order of magnitude as the 3 disputed rows.

Fourth, accuracy hides the differences between data problems. All nine interventions move accuracy by less than 0.7 percentage points, while spam recall ranges across 4.95 points (90.91% to 95.86%). The mechanisms also differ: masking interventions move only precision, row-removal interventions move only recall. Under a 13.4% class imbalance, reporting accuracy alone compresses very different data defects into the same "almost no effect" appearance.

One principle runs through the whole audit: **automated signals only retrieve candidates; the final decision always adds a human policy judgement.** Rejected candidates are recorded for every target, and a small number of false positives are kept on purpose so that the calibration can be checked rather than taken on trust.

---

2. Data and Method

2.1 Provenance and Integrity

The raw corpus is the UCI SMS Spam Collection, downloaded and verified by script. The archive SHA-256 is `1587ea43…3ce3` and the extracted raw file SHA-256 is `7d039a24…c239d`. The join against the fixed course manifest preserves the official split and does not regenerate any row assignment.

The canonical working table `data/canonical_sms.csv` is read through `load_canonical()` with `keep_default_na=False`. Without that setting, pandas converts the literal strings `NA`, `null`, and `nan` into missing values. Those strings appear in real messages in this corpus, so they are valid text, not missing-data markers.

Integrity check	Result
Total rows	5,574
Training / held-out rows	4,460 / 1,114
Unique IDs	5,574
Unique UCI row numbers	5,574
Empty text rows	0
Decoded encoding	utf-8

2.2 Dataset Profile

Class balance. Spam is 13.40% of the full corpus. The training and held-out shares are 13.41% and 13.38%, so the label prior is nearly identical on both sides and the fixed split shows no visible shift.

Split	ham	spam	Total	Spam share
train	3,862	598	4,460	13.41%
heldout	965	149	1,114	13.38%
All	4,827	747	5,574	13.40%

The class imbalance sets up every metric discussion that follows: a classifier that always answers ham already scores 86.62% accuracy. This report therefore uses **spam recall, spam F1, and macro-F1** as its main criteria, with accuracy as secondary.

Surface cues. Spam and ham differ sharply in length, digits, contact details, and promotional wording.

Measure	ham	spam
Median characters	52	149
Median digit count	0	16
Contains URL	0.29%	18.47%
Contains phone-like number	0.10%	60.78%
Contains currency or rate cue	0.62%	38.15%
Contains promotional token	9.16%	87.55%

This is a descriptive fact, not by itself a claim that any label is wrong. It motivates the shortcut audit in Section 5, which asks whether these cues explain too much of the performance.

Split comparability. On surface measures other than the label, training and held-out data are close: median characters 62 versus 60, URL rate 2.69% versus 2.87%, promotional-token rate 19.91% versus 18.67%. The clearest difference is a slightly higher rate of phone-like numbers in the held-out set (8.07% versus 8.89%), which is worth remembering when reading spam recall.

Corpus artifacts. Spam prevalence by original UCI row-number decile ranges from 11.67% to 15.80%, so the corpus is not ordered by label. 309 rows (5.54%) contain HTML escape sequences and 483 rows (8.67%) contain non-ASCII characters. These are kept rather than cleaned, so that every downstream audit runs against the same public text.

2.3 Reference Baseline

The course starter package supplies the audit protocol, the split manifest, a data dictionary, and a format sample. It contains no model, and the handout requires teams to write their own profiling, baseline, and audit code. Every mention of "the baseline" in this report therefore refers to the reference baseline described below. The only component fixed by the course is the split itself.

The baseline is a linear text classifier built from **TF-IDF features and logistic regression**, trained in two views:

- **Word model** (the primary baseline): TF-IDF over words and bigrams (`ngram_range=(1,2)`, `min_df=2`, `sublinear_tf=True`), a 12,540-dimensional vocabulary, and logistic regression with `C=4.0`, `solver='liblinear'`, `random_state=42`.
- **Character model** (a secondary signal source): TF-IDF over character 3-to-5-grams (`analyzer='char_wb'`, capped at 50,000 features), with the same logistic regression settings. The two models differ only in how text is split into features, not in algorithm.

A linear model was chosen for audit reasons rather than for accuracy. Every prediction can be decomposed back into specific tokens and coefficients, which is what makes the claim "the model disagrees with this label" something a human reviewer can check.

Training rows are scored with **5-fold out-of-fold prediction**. If the model were fitted on all training data and then used to predict that same data, it would have memorised each label, and "the model disagrees with this label" would carry no information. Out-of-fold scoring guarantees that the model scoring each row has never seen that row, which is a precondition for Target 4. Held-out rows are predicted by a model fitted on the full training set.

Condition	n	Accuracy	Spam precision	Spam recall	Spam F1
Word held-out (reference baseline)	1,114	**99.1023%**	0.9929	0.9396	0.9655
Character held-out	1,114	98.7433%	0.9927	0.9128	0.9510
Word out-of-fold (train)	4,460	98.2063%	0.9924	0.8729	0.9288

The highest-weighted spam features in the word model are `call` (+6.69), `txt` (+6.65), `text` (+5.54), `free` (+4.62), `mobile` (+4.43), `claim` (+4.41), and `www` (+4.33). The most negative ham features are `my` (−3.28), `me` (−3.12), `that` (−2.76), and `ok` (−2.69). The spam side is dominated by promotional verbs and channel words, the ham side by first-person pronouns. This distribution already points toward the shortcut finding in Section 5.

One anomaly needs explaining. Held-out spam recall (0.9396) is 6.7 percentage points higher than the out-of-fold estimate on the same distribution (0.8729). The held-out set was never used for tuning, so its score should not exceed a cross-validated estimate. Sections 3.3 and 7.2 explain the gap: the held-out set contains rows the model already saw during training.

2.4 Method: Separating Retrieval from Adjudication

All six targets follow the same structure. **Automated signals retrieve candidates from the 5,574 rows; a separate human policy review decides what is real.** The two stages write to separate files, and rejected candidates are kept alongside accepted findings.

Target	Retrieval method	Basis for decision
1 Exact duplicates	Group by normalised text (lowercase, collapsed whitespace)	Objective string rule, no human step
2 Near duplicates	TF-IDF cosine ≥ 0.92 plus threshold sensitivity analysis	Human accept/reject on each pair
3 Cross-split leakage	Apply Targets 1 and 2 across the split, plus row-position check	Inherits the decisions from 1 and 2
4 Label errors	Word/character out-of-fold scores plus neighbours, requiring ≥2 independent signals	Signals plus reading the message
5 Shortcut features	Shallow probe on derived features versus same-algorithm full-text control	Standardised coefficients and share of performance
6 Ambiguity	Model disagreement, boundary distance, or mixed neighbour labels (any one)	Annotation-policy reading

The direct benefit of this separation is that retrieval thresholds can be set per target according to how strong the resulting claim is, instead of forcing one global threshold that costs either recall or precision (see Section 6.1).

---

3. Targets 1–3: Duplicates and Cross-Split Leakage

3.1 Exact Duplicates

Exact duplication follows the public rule in the protocol: strings are identical after lowercasing and collapsing whitespace. After normalisation the corpus contains 5,159 unique texts, meaning 415 rows are repeats beyond a first occurrence.

Measure	Result
Exact duplicate clusters	290
Rows in those clusters	705
Clusters crossing the split	5
Clusters with a label conflict	0
Largest cluster size	30

No exact duplicate cluster contains a label conflict. There is no case where the same text carries different labels. This means the annotation is self-consistent at the level of literal repetition, which is a positive quality finding.

Findings are reported **by cluster**, one representative row per cluster, rather than listing all 705 members. Confidence and severity are recorded separately. The evidence for an exact duplicate is string identity, which is mechanically verifiable and leaves no room for a false positive, so all 290 clusters carry high confidence. Severity is carried by `recommended_action` instead: the 5 clusters that straddle the split are routed to `rebuild_split_by_cluster` because they threaten evaluation integrity, while the remaining 285 are routed to `dedupe_by_cluster` as ordinary cleanup. Section 8 sets out this convention in full.

3.2 Near Duplicates

Candidates are retrieved using both the word and character TF-IDF views, scored by `max(word_cos, char_cos)`, with a retrieval floor of 0.88 and the protocol decision threshold of 0.92. To make the threshold choice auditable rather than hidden tuning, we ran a sensitivity analysis across 0.88 to 0.96 and recorded how the candidate count and cross-split count change at each step.

At 0.92 the search returns 365 candidate pairs. After excluding 17 heldout-to-heldout pairs, which cannot affect evaluation integrity, 348 pairs went to human review:

Review outcome	Pairs	Cross-split
Accepted as near duplicates	**200**	85
Rejected as false positives	**148**	5
Undecided	0	—

The 148 rejections fall into two groups: pairs that **share only a generic phrase or greeting** with no specific content anchor, and pairs that are **textually similar but different in meaning**. This reflects a deliberate evidence standard: the same generic phrase counts as an **exact** duplicate, because that is an objective string fact, but does not count as a **near** duplicate. A near duplicate must point to the same source, template, or specific content.

3.3 Cross-Split Leakage

Combining exact duplicates with accepted near duplicates across the split, **63 held-out rows** can be recovered from training data (5 high, 46 medium-high, 12 medium):

Leakage channel	Unique held-out rows affected
Exact duplicate across split	5
Accepted near duplicate across split	60
Combined, deduplicated	**63**

That is **5.7%** of the held-out set answering questions the model has already seen. Section 7.2 measures what this is worth in score terms.

Row position is not a leakage channel. A probe using only the normalised UCI row number reaches 86.62% accuracy and 0.00% spam F1, identical to the majority-class baseline (Section 5.2). This rules out any label leak through corpus ordering.

4. Target 4: Suspected Label Errors

4.1 Method

The protocol defines a label error as a case where at least two independent signals disagree with the assigned label. We compute three signals for all 5,574 rows:

Signal	Source	Why it is independent
Word disagreement	Word/bigram TF-IDF logistic regression	Training rows scored out-of-fold, so the model never saw the row
Character disagreement	Character 3-to-5-gram logistic regression	Separate feature space; catches obfuscated spellings such as `fr33` and `w1n`
Neighbour disagreement	Character TF-IDF nearest neighbours	Local label structure, independent of any decision boundary

The candidate threshold takes the protocol definition strictly — **at least two signals must disagree** — which yields 70 candidates. The neighbour window **excludes any neighbour whose normalised text is identical** to the target row. Without that exclusion a message could have its own duplicate copy confirm its label, and the evidence would be self-contaminating (see Difficulty 3 in Section 9).

Confidence follows a reproducible rule rather than a judgement made after the fact:

> **high** — opposition strength ≥ 0.95, or all three signals disagree **medium** — two signals disagree with strength below 0.95 **low** — fewer than two automated signals; the claim rests on policy reading alone

4.2 Findings

We submit **15 suspected label errors** (5 high, 9 medium, 1 low). All of them argue in the same direction: the public label is spam and we propose ham. 13 are training rows, which feed the overlay experiment, and 2 are held-out rows, which are **flagged only and never relabelled**, as the handout requires.

ID	Split	Confidence	Opposition	Signals	Reading
T2378	train	high	0.992	3/3	Observational joke about driving, no solicitation
H0143	heldout	high	0.982	2/3	One-line joke about tattoos, flagged only
T0059	train	high	0.965	2/3	"Divorce Barbie" joke
T2232	train	high	0.963	2/3	Child-versus-teenager joke
H0896	heldout	high	0.916	3/3	Recipient asking what to do after winning, flagged only

One policy pattern is worth naming on its own: **T1798, T4296, and T1174 discuss, quote, or complain about spam rather than send it.** They look like spam because they repeat promotional wording, but they are ordinary personal messages. T1174 has only one supporting signal, making it the weakest claim in the set, and it is reported as low.

4.3 Rejected Candidates and False-Positive Control

Of the 45 top-ranked candidates read by hand, **31 were adjudicated** `keep_but_flag`: the model is wrong and the public spam label stands. The rejections cluster into four patterns:

Pattern	Examples	Evidence that beats the model
Tariff disclosed at the end	T2755, T3105, T0006	`one gbp/sms`, `cst std ntwk chg £1.50`
Reply-keyword mechanism	H0814, T3202	`Send A, B or C`, short codes
Adult-content solicitation	T1505, T1926	Instructions such as `Txt XXX SLO(4msgs)`
Legitimate commercial broadcast	T2229, T2273	Burger King and Interflora promotions

T2755 shows why rejection cannot be automated. Its opposition strength of 0.953 is already above the high threshold, but the message ends with the tariff disclosure `one gbp/sms`. Without reading the text, it would have been filed as a high-confidence label error. This is exactly the shape of case a false-positive trap in the hidden audit key is designed to catch.

All 80 adjudicated candidates, with reasons, are recorded in `adjudication_memo.csv`. Their distribution is `keep_but_flag` 49, `should_fix` 15, `ambiguous_policy_case` 14, `false_positive_audit_finding` 2.

---

5. Target 5: Shortcut Features

5.1 Method

The shallow probe receives **only 12 derived numeric features and never receives word order**: character count, token count, average token length, digit count, digit ratio, uppercase ratio, exclamation count, currency-marker count, URL presence, phone-like-number presence, promotional-token count, and, tested separately, normalised row position. Numeric features are standardised before entering the logistic regression.

The control is what makes this section's conclusion valid: **the full-text control uses the same algorithm (logistic regression)**, so the feature set is the only variable. If the control used a different model family, any performance gap would be attributable to the model rather than the features, and the argument would collapse. The full-text result is read from the frozen shared held-out predictions rather than retrained.

5.2 Results

Feature set	Accuracy	Spam precision	Spam recall	Spam F1
Majority class (always ham)	86.62%	0.00%	0.00%	0.00%
Row position only	86.62%	0.00%	0.00%	0.00%
Shape only (length, case, exclamations)	89.50%	70.00%	37.58%	48.91%
Contact and promotion only	**98.11%**	95.07%	90.60%	**92.78%**
All shallow features	98.20%	95.10%	91.28%	93.15%
All shallow plus row position	98.20%	95.10%	91.28%	93.15%
Full text with 15 promotional tokens masked	98.92%	97.90%	93.96%	95.89%
Full text (reference baseline)	99.10%	99.29%	93.96%	96.55%

Main result: contact and promotional cues alone reach 98.11% accuracy and 92.78% spam F1, about one percentage point below the full word model's 99.10% and 96.55%. Most of what makes this dataset separable is carried by shallow surface cues, and a model can approach the ceiling without representing sentence meaning.

The protocol's 70% accuracy warning threshold does not discriminate on this dataset, since the majority-class classifier already reaches 86.62% while detecting no spam at all. The informative evidence is that the gap between the shallow probe and the full model is compressed to roughly one point.

Row position alone has no predictive value. Its accuracy and spam F1 match the majority-class baseline exactly. This is a clean negative result that rules out a row-order shortcut.

5.3 Which Cues Matter

The standardised coefficients answer this directly, with positive values pointing toward spam: `digit_count` (+2.566), `promo_token_count` (+1.723), `avg_token_length` (+0.760), `has_url` (+0.678), `currency_count` (+0.642), `exclamation_count` (+0.336). Because the inputs are standardised, these magnitudes are comparable within the same fitted probe.

5.4 Interpretation and Limits

Digits, prices, URLs, phone numbers, and promotional wording are **legitimate** evidence of spam, so their predictive value is not leakage in itself. They count as shortcuts because they **work without sentence meaning and fail under distribution shift**. An ordinary discussion of prices triggers a false positive: H0946, a genuine conversation about apartment rent, is pushed toward spam by eleven digits and three currency markers. A solicitation without the usual numeric markers is missed: H0044, a ringtone-club offer, has almost no digits or contact cues, so the shallow probe misses spam that the text model detects.

Coefficients describe association, not causation. The 8 example rows in this section are stress cases and are not requests to change held-out labels. Rows that turned out to be annotation-policy cases, such as H0143 and H0896, were handed to Target 4 for joint adjudication and are not double-counted as shortcut evidence here.

---

6. Target 6: Ambiguous Examples and the Score Ceiling

6.1 Retrieval: Three Questions, Any One Qualifies

We are looking for messages where **both ham and spam are defensible labels**. This property cannot be decided by a rule, so three independent questions retrieve suspects and every retrieved row is then read by hand:

1. **Do the two models disagree?** The word model says ham and the character model says spam, or the reverse. → 45 rows
2. **Is the model itself hesitating?** The predicted probability falls between 0.32 and 0.68. → 101 rows
3. **Are the neighbour labels mixed?** Among the most textually similar messages, the labels are close to an even split, which means **annotators were already inconsistent on this kind of message**. → 100 rows

The three questions fire 246 times in total but collapse to **177 unique candidates** (144 training, 33 held-out). The overlap is partial, which confirms that the three questions capture partly different phenomena rather than asking the same thing three ways.

The third question deserves separate attention. The first two ask **what the model thinks**; only the third asks **what the original annotators did**. An even split among neighbour labels is direct evidence that the annotation guideline itself is ambiguous, and it has nothing to do with how strong the model is.

Why use OR here when Target 4 uses AND? Because the strength of the resulting claim differs. "This row is mislabelled" asserts that the annotation is *wrong*, which is a strong claim, so the retrieval stage must be strict and accept missed cases. "This row is arguable" only says the row is *worth reading*, so a wide net costs little, because every retrieved row is reviewed by hand anyway. The retrieval threshold should match the strength of the claim rather than being uniform across targets.

The candidate table carries an `ambiguity_score` that combines the three questions into a single ranking value. It **only determines reading order and takes no part in any decision**.

6.2 Adjudication: Ambiguity Is Not Low Confidence

"Ambiguous" is reserved for one situation: **without knowing who sent the message, whether the user had subscribed, or whether it belongs to a marketing campaign, both ham and spam are defensible.**

This is different from "the model is unsure". An unsure model may simply be a weak model. Ambiguity means **the information itself is insufficient to decide**, and a stronger model would not resolve it either. Reporting boundary-region rows as ambiguous would mean measuring the classifier instead of the dataset.

The direct evidence for this distinction: of the 14 confirmed ambiguity findings, **10 came from the label-error candidate pool rather than the boundary pool**. The two strongest, T2139 (opposition 0.987, 3 of 3 signals) and T2710 (0.951, 3 of 3), are cases where **the model is not hesitating at all**, yet they remain annotation-policy problems. The two criteria are orthogonal.

All 177 candidates were read. Only **14** were confirmed as ambiguous, a rate of 7.9%. Most of the rest turned out to be cases where the model is genuinely uncertain but the annotation is not in dispute, which is a model limitation rather than a dataset defect.

6.3 Findings

We submit 14 ambiguity findings (13 medium, 1 low). The recurring policy boundaries are:

- **The 070 number pair (T2710 and T3390).** The UK 070 range is both a legitimate personal follow-me range and a range abused by premium redirect services. Neither message discloses a tariff, and both are judged ambiguous under the same standard.
- **Possibly subscribed solicitations (T2847, a club ticket; T3981, a UNICEF donation appeal).** Membership lists and charity exemptions are real policy carve-outs. They contrast with T2229 and T2273, pure commercial broadcasts with no consent story, which were rejected and confirmed as spam. Together the two groups mark the two sides of this boundary.
- **Context that cannot be recovered (T0774, truncated text; H0550, a corpus remnant).** The text is no longer sufficient to decide, which is itself a finding about corpus quality.

6.4 False Positives Kept on Purpose

Two rows are submitted at low confidence and explicitly marked as rejected candidates, to demonstrate that prediction uncertainty is neither a label error nor semantic ambiguity:

ID	Text	Why it is not a finding
H0935	"I liked the new mobile"	Probability sits near 0.5 and neighbour labels are mixed, but the content is ordinary ham
T0470	"Waiting for your call."	Word probability 0.689 comes from lexical overlap with call-to-action spam, but**no neighbour disagrees with the label**

Keeping these two has a cost, since they dilute the apparent precision of the finding set. That cost is what makes the confidence distribution meaningful.

6.5 How Ambiguity Limits the Score Ceiling

The handout's question for Target 6 has two halves. The second half asks how ambiguous examples limit the score ceiling. This section answers it directly.

The reference baseline makes **10 errors** on the 1,114 held-out rows. Classified by audit status:

Status	Count	Rows
Genuine model errors	7	H0287, H0318, H0330, H0461, H0814, H0903, H0927
Suspected public label errors	2	H0143, H0896
Ambiguous policy cases	1	H0550

3 of the 10 errors are data problems rather than model failures. This limits the score ceiling in three ways.

First, there is an error floor no model can cross. A genuinely ambiguous example has, by definition, no single correct label. Whichever side the model predicts, the label it is scored against is an arbitrary convention. Such rows are **irreducible error**. This gives a computable ceiling: even a perfect model that fixed all 7 genuine errors would still be marked wrong on the 3 data-problem rows.

```
ceiling = (1114 − 3) / 1114 = 99.7307%
```

With the data as it stands, **no model can exceed 99.73% held-out accuracy**.

Second, the remaining headroom and the disputed region are now the same size.

	Rows	Share of held-out
All remaining headroom above 99.1023%	7	0.63%
Rows with disputed annotation (ambiguous plus suspected errors)	3	0.27%
Ratio	≈ 2.3 : 1	

This means **any improvement smaller than about 0.3 percentage points cannot be distinguished from re-adjudicating a few annotation decisions**. The benchmark has entered a noise-dominated regime: the second decimal place no longer measures model capability, it measures annotation convention. Section 7.2 confirms this experimentally — moving just 4 rows out of scoring shifts the headline number from 99.1023% to 99.3694%.

Third, the training set fixes the convention into the model. 13 of the 14 ambiguity findings are training rows. They do not affect the held-out score directly, but they **teach the model an arbitrary annotation convention**, such as "any 070 number is spam". The model learns the convention rather than a generalisable rule, and because the evaluation set follows the same convention, that bias is rewarded rather than penalised. Section 7.3 shows this mechanism directly: correcting training labels *lowers* the score.

In summary, ambiguous examples limit the ceiling in three ways: they set a hard upper bound of 99.73%, they compress the remaining headroom to the same order of magnitude as the disputed region so that small gains become uninterpretable, and they fix arbitrary conventions into the model through the training set. Continuing to optimise for score on this dataset no longer measures model quality.

---

7. Comparison of Data Interventions

The handout requires comparing at least two data interventions. This report runs **nine**, covering all six audit targets, cross-validated by two independent implementations (Section 7.5).

Two constraints hold throughout: **the canonical table is never modified** (every intervention is expressed as an overlay or a derived copy), and **held-out labels are never changed**.

7.1 Intervention Matrix

Interventions are grouped by **which part of the data they touch** rather than by audit target, because that axis explains why each one raises or lowers the score.

Intervention	What changes	Size	Retrain	Accuracy	Spam recall
Reference baseline	—	—	—	99.1023%	93.9597%
Evaluation side (model frozen)					
Leakage-aware evaluation	Rows removed from scoring	−63	No	99.0485%	**90.9091%**
Adjudication-aware evaluation	Rows removed from scoring	−4	No	**99.3694%**	**95.8621%**
Training side (retrained)					
Training-label overlay	Training labels changed	13 relabelled	Yes	98.9228%	92.6174%
Remove leakage partner rows	Training rows deleted	−71	Yes	99.0126%	92.6174%
Exclude ambiguous training rows	Training rows deleted	−12	Yes	99.0126%	92.6174%
Deduplicate by cluster	Training rows deleted	−417	Yes	98.8330%	92.6174%
Promotional-token masking (15 tokens)	Text rewritten	All rows	Yes	98.9228%	93.9597%
Full cue masking	Text rewritten	All rows	Yes	98.7433%	93.9597%
Combined					
Combined training view	All training-side changes	4,460 → 3,982	Yes	98.4740%	91.9463%

The key to reading this table: every intervention moves accuracy by less than 0.7 points, while spam recall spans 90.91% to 95.86% — a range seven times wider. Section 7.6 develops what this means.

7.2 Evaluation-Side Interventions: Change the Measurement, Not the Model

These two interventions share one mechanism — **freeze the baseline predictions and change only which rows are scored** — but point in opposite directions, which makes them a useful pair.

Leakage-aware evaluation (Target 3)

Held-out rows recoverable from training (Section 3.3) are removed from scoring. The intervention is graded, to separate the two leakage channels:

Condition	Rows removed	n	Accuracy	Spam recall
Full held-out (reference baseline)	0	1,114	99.1023%	93.9597%
Remove exact-duplicate leakage only	5	1,109	99.0983%	93.7931%
Remove exact plus accepted near-duplicate leakage	**63**	1,051	99.0485%	**90.9091%**

The grading reveals something the total hides: almost all the damage comes from near duplicates, not exact ones. Removing the 5 exact-leakage rows costs only 0.17 points of spam recall. Removing the further 58 near-duplicate rows costs 2.88 points. An audit that stopped at exact string matching would have concluded that leakage is negligible. The 348 manually reviewed pairs in Section 3.2 are justified by this table.

The baseline predicts **all 63 removed rows correctly** (63 of 63), because it saw them during training. 50 of them are spam, and they are what lifts the true 90.91% spam recall to the reported 93.96%. This is the single largest effect among all interventions.

Adjudication-aware evaluation (Targets 4 and 6)

Held-out rows the audit judged **ambiguous** (no single correct answer) or **suspected mislabelled** (scored against a label we believe is wrong) are removed. Neither kind is a fair test of the model.

Condition	Rows removed	n	Accuracy	Spam recall
Full held-out (reference baseline)	0	1,114	99.1023%	93.9597%
Remove ambiguous rows (H0030, H0550)	2	1,112	99.1906%	94.5578%
Also remove suspected label errors (H0143, H0896)	**4**	1,110	**99.3694%**	**95.8621%**

One further held-out row marked ambiguous, **H0935, is deliberately not removed.** It is the false positive retained in Section 6.4, adjudicated as an ordinary uncontroversial row. Having judged that it is not ambiguous, we cannot then use ambiguity as grounds to remove it from scoring. This is a consistency check on the adjudication.

Half of this intervention is circular by construction, and that needs stating. Three of the 4 removed rows were already errors for the baseline, but the two groups differ in kind:

Removed rows	Baseline wrong	Circular with retrieval
Suspected label errors (H0143, H0896)	2 of 2	**Yes**
Ambiguous (H0030, H0550)	1 of 2	No

The label-error rows were retrieved precisely because "at least two model signals disagree with this label". On held-out data, "the model disagrees with this label" and "the model got it wrong" are the same event. Removing them must raise the score. That is not an experimental result, it is the same fact restated. **The rise from 99.19% to 99.37% therefore carries no evidential weight, and this report does not rely on it.**

The informative part is the ambiguous rows: the model answered one of them (H0030) correctly. That group was selected by reading the messages and applying policy judgement, independently of whether the model was right. The result shows that ambiguous rows are **not a synonym for rows the model fails on**, which supports the argument in Section 6.2 from the data side. With n = 2 this quantifies a mechanism rather than an effect size; the ceiling argument in Section 6.5 rests on the hard upper bound, not on these 0.09 points.

7.3 Training-Side Interventions: Change the Data and Retrain

Training-label overlay (Target 4)

The 13 training-label corrections from Section 4 are applied as an overlay file, without overwriting the public labels. The word baseline is retrained with `SEED=42` and evaluated on the full held-out set.

Condition	Accuracy	Spam recall
Public labels (reference baseline)	99.1023%	93.9597%
Overlay applied	98.9228%	92.6174%

The score drops, but this does not mean the intervention was wrong. What we corrected is that the public labels marked several jokes and quoted messages as spam, while the held-out set is still scored under **the same public convention**. Correcting the training set therefore costs the model points on an evaluation set that keeps the old convention. This is the clearest available demonstration that **when the evaluation set carries annotation noise, the benchmark score penalises data corrections that move in the right direction.**

Promotional-token masking and full cue masking (Target 5)

Section 5 showed that the model relies heavily on surface promotional cues. Two masking scopes were applied, forming a gradient:

Condition	Masking scope	Accuracy	Spam precision	Spam recall	False positives
Full text (reference baseline)	—	99.1023%	0.9929	0.9396	1
Promotional-token masking	The fixed 15-token lexicon	98.9228%	0.9790	0.9396	3
Full cue masking	URLs, phones, currency and rate markers, digit runs, promotional lexicon	98.7433%	0.9655	0.9396	5

Spam recall is identical across all three conditions (0.9396). The entire difference falls on precision (0.9929 → 0.9790 → 0.9655).

This tells us more than any single condition would. As masking becomes more aggressive, the model **does not miss more spam**; it **misclassifies more ham**. Surface cues are therefore not necessary for detecting spam — remove one group and the model falls back smoothly onto another, such as short codes and formatting structure. **The shortcut is not a single bridge but a dense, redundant network of surface cues.** This also explains why contact and promotional features alone reach 98.11% in Section 5.2.

Three row-removal interventions

Intervention	Rows removed	Training rows	Accuracy	Spam recall
Reference baseline	—	4,460	99.1023%	93.9597%
Remove leakage partner rows	Training rows duplicating held-out rows	4,389 (−71)	99.0126%	92.6174%
Exclude ambiguous training rows	Rows adjudicated as ambiguous policy cases	4,448 (−12)	99.0126%	92.6174%
Deduplicate by cluster	Keep only the earliest row per duplicate cluster	4,043 (−417)	98.8330%	92.6174%

All four conditions land on exactly 92.6174% spam recall, while the deletion size ranges from 12 rows to 417. Deleting 417 rows, nearly 10% of the training data, has the same effect as deleting 12. This means **the deleted rows are overwhelmingly redundant copies from which the model learned nothing extra**, which confirms the duplicate finding in Section 3.1 from the opposite direction: those 415 surplus copies really are surplus.

It is also worth recording that removing leakage partners (−71) and excluding ambiguous training rows (−12) produce cell-for-cell identical confusion matrices (tn 965, fp 0, fn 11, tp 138). Deleting two entirely different sets of training rows yields the same held-out decisions, which further supports the reading above: neither set carries decisive training signal, and the decision boundary does not depend on them.

7.4 Combined Intervention

Stacking all training-side interventions — remove leakage partners, deduplicate by cluster, exclude ambiguous rows, apply the remaining label overlay, and mask cues — reduces the training set from 4,460 to **3,982 rows**:

Condition	Training rows	Accuracy	Spam recall
Reference baseline	4,460	99.1023%	93.9597%
Combined training view	3,982	**98.4740%**	**91.9463%**

This is the largest drop of any intervention (−0.63 points accuracy, −2.01 points spam recall), but **it is not evidence that the data got worse**. It is a conservative stress test, not a proposed replacement dataset. On an evaluation set that itself contains leakage, carries annotation noise, and rewards surface cues, **a cleaner training set producing a lower score is the expected outcome** — this is the extreme form of Observation 3 in Section 7.6. Its value is in showing the cumulative magnitude of the data-quality problems: each one is modest alone, but together they clearly weaken the claim that the headline metric represents clean, independent generalisation.

7.5 Cross-Validation

The results in this section come from two independent implementations: the single-target scripts (`label_overlay_experiment.py`, `intervention_leakage_eval.py`, `intervention_ambiguity_eval.py`) and the unified batch script `run_impact_interventions.py`. The three shared conditions agree to the last digit:

Condition	Both implementations
Reference baseline	accuracy 0.991023 / spam recall 0.939597, identical
Leakage-aware evaluation	accuracy 0.990485 / spam recall 0.909091, identical
Training-label overlay	accuracy 0.989228 / spam recall 0.926174, identical

Two independent implementations producing the same numbers is a substantive check on pipeline correctness, and stronger than a single implementation checking itself.

7.6 Overall Reading

Intervention	Accuracy	Spam recall	Spam precision
Leakage-aware evaluation	−0.05 pt	**−3.05 pt**	−0.39 pt
Adjudication-aware evaluation	+0.27 pt	+1.90 pt	−0.01 pt
Training-label overlay	−0.18 pt	−1.34 pt	−0.01 pt
Promotional-token masking	−0.18 pt	**±0**	−1.39 pt
Full cue masking	−0.36 pt	**±0**	−2.74 pt
Row removal (three variants)	−0.09 to −0.27 pt	−1.34 pt	+0.7 to +1.4 pt
Combined training view	−0.63 pt	−2.01 pt	−2.81 pt

Observation 1: leakage hands the model 63 questions it has already seen. The baseline answers all 63 removed held-out rows correctly, and the 50 spam rows among them are what lift true spam recall from 90.91% to 93.96%. This is the largest single effect measured.

Observation 2: accuracy compresses mechanically different problems into one appearance. All nine interventions move accuracy by less than 0.7 points, while spam recall spans 4.95 points with unrelated patterns — masking moves only precision, row removal moves only recall, and adjudication-aware evaluation moves in the opposite direction entirely. **If only accuracy were reported, these nine data problems would look identical, all "almost no effect", when in fact their mechanisms have nothing in common.** This is the central methodological conclusion of the report: under a 13.4% class imbalance, accuracy is not a neutral summary statistic but one that systematically erases differences.

Observation 3: the benchmark score is distorted in two opposite directions at once. Leakage-aware evaluation shows it is **inflated** (some rows are free points). The training-label overlay and the combined view show it **penalises correct data repairs** (the evaluation set keeps the same flawed convention). The gain from adjudication-aware evaluation is mostly an accounting effect rather than new evidence (Section 7.2). Taken together: 99.10% is neither an upper bound on true model capability nor a target that improves monotonically as the data gets cleaner. **It is the result of several data defects pointing in opposite directions and partly cancelling out.**

7.7 Interventions Proposed but Not Executed

For duplicate findings, `suspicious_examples.csv` recommends the actions `dedupe_by_cluster` and `rebuild_split_by_cluster`, but **this project does not execute them**. Both would change the train/held-out split, and the split manifest is a fixed course contract; changing it would make results incomparable with the baseline. They are listed as recommendations in Section 10 rather than as completed experiments.

---

8. Submission Composition and Uncertainty Calibration

8.1 What the Two Files Do

The audit produces two files answering different questions. `suspicious_examples.csv` answers **which rows have problems**. `adjudication_memo.csv` answers **what decision each candidate received and on what grounds**. The memo records rejected candidates as well as accepted ones; rejected candidates exist only

there, which is what makes the audit process reviewable.

`suspicious_examples.csv` contains **670 rows** in the protocol's 9-column schema, blocked by issue type and ranked within each block by evidence strength:

Target	issue_type	Rows	high	medium	low
1	`exact_duplicate`	290 (one per cluster)	290	—	—
2	`near_duplicate`	278	—	127	151
3	`leakage`	63	5	58	—
4	`label_error`	15	5	9	1
5	`shortcut`	8	3	5	—
6	`ambiguous`	16	—	13	3
	Total	**670**	**303**	**212**	**155**

All three protocol constraints are satisfied: 670 rows ≥ 35, all six issue types present, and high confidence at **45.22% ≤ 55%**.

Two notes on reading the table. First, `exact_duplicate` is reported by cluster: 290 rows represent 290 clusters covering 705 member rows, rather than listing members individually. Second, of the 16 `ambiguous` rows, **14 are genuine ambiguity findings** (13 medium, 1 low); the other two, H0935 and T0470, are the false positives kept on purpose from Section 6.4. They are submitted at low confidence and marked `reject_audit_finding`. They appear in the submission so the calibration can be checked, not as claims that the rows are problematic.

`adjudication_memo.csv` contains **444 rows**, consolidating three review ledgers and mapping them onto the four adjudication categories defined by the protocol:

Source	Rows	Targets covered
Label-error and ambiguity memo	80	4, 6
Near-duplicate pair review	348	2
Shortcut candidate review	16	5

Decisions break down as `should_fix` 215, `false_positive_audit_finding` **158**, `keep_but_flag` 57, `ambiguous_policy_case` 14.

8.2 What Confidence Means Here

Confidence answers **how certain we are that a row really belongs to the issue type it is filed under**. Severity is carried separately by `recommended_action`. Keeping them apart is deliberate: a cross-split duplicate cluster and a within-split one rest on equally certain evidence (string identity in both cases), but the first threatens evaluation integrity while the second is routine cleanup. That difference belongs in the recommended action, not in an artificially lowered confidence value.

Confidence	Criterion	Typical source
high	Mechanically verifiable evidence, or opposition ≥ 0.95 / all three signals disagreeing	Exact duplicate clusters, leakage backed by exact duplicates
medium	Objective evidence that still required judgement	Accepted near-duplicate pairs, two-signal label errors, ambiguous policy cases
low	The weakest retained claims, including deliberately kept false positives	Single-signal label errors, generic-phrase near duplicates, control cases

Under this convention all 290 exact duplicate clusters are high, since string identity leaves no room for a false positive. High confidence therefore reaches 45.22%, still within the protocol limit.

8.3 The Cost of Calibration

The protocol's 55% cap is a deliberate calibration constraint: it makes "mark everything high" impossible and forces the finding set to express internal gradations. But the real evidence of calibration is not that ratio. It is the three decisions we made that reduce the apparent size of the finding set:

1. **148 near-duplicate pairs rejected as false positives** — they share only generic phrasing, or are textually similar with materially different meaning.
2. **31 label-error candidates returned as `keep_but_flag`** — the model is wrong and the public spam label stands.
3. **2 rows submitted at low confidence purely as controls** (H0935, T0470), demonstrating that prediction uncertainty is not a finding.

Together these exclude or downgrade 181 candidates, all recorded with reasons in `adjudication_memo.csv`. The basis is the handout's requirement in Section IV: a long list of weakly supported high-confidence findings is worth less than a ranked list that clearly explains its uncertainty and its rejected candidates.

The other side of the trade-off should also be stated. Exact and near duplicates are verifiable facts with very low false-positive risk, so we chose to report them completely rather than sample them. This raises coverage of known issues without sacrificing calibration: of the 670 submitted rows, only 303 make a strong claim, and every one of those rests on evidence that can be checked mechanically.

---

9. Difficulties and Solutions

Difficulty 1: Making a Pipeline That Contains Human Judgement Fully Reproducible

The problem. The near-duplicate review involves human accept/reject decisions on 348 candidate pairs. Those decisions must map stably onto the automatically retrieved candidate set. If the mapping depends on any identifier produced at run time, the pipeline cannot be reproduced: on a re-run the decisions and the candidates drift out of alignment, and the downstream leakage conclusion changes with them. This was the most substantial engineering challenge in the project, because it involves three separate layers:

Layer	Source of risk	Design adopted
1	Pair identifiers derived from traversal or insertion order drift with the implementation	Sort by `(id_a, id_b)` before numbering, decoupling identifiers from traversal order
2	Human decisions embedded in script constants are bound to those drifting identifiers	Decisions moved into a separate data file, indexed by `pair_key = "|".join(sorted([id_a, id_b]))`
3	**The candidate set itself must be deterministic**	Replaced top-K nearest-neighbour retrieval with a full pairwise cosine threshold scan

The third layer is the least obvious. Even with the first two handled, top-K retrieval through `NearestNeighbors` returns an arbitrary choice among neighbours tied on cosine value, so the candidate **set** can differ between runs. In practice two pairs varied: `T1545|T3892` and `T3609|T3892`. Computing pairwise cosine similarity over all examples and taking the upper triangle above the threshold removes any dependence on traversal order.

Outcome and generalisation. Running the three scripts in sequence twice now produces byte-identical outputs. The generalisable lesson is that **human judgement is data, not code**. Writing it into a script constant binds it to a temporary identifier that drifts with the implementation. The same tie-breaking issue appeared in the top-8 display tables used for reporting, where many pairs share `max_cosine = 1.0000`; those are now sorted with `(id_a, id_b)` as a secondary key.

Difficulty 2: Separating Semantic Ambiguity from Model Uncertainty

The problem. The easiest approach to Target 6 would be to mark everything near the decision boundary as ambiguous. That measures the classifier's certainty rather than the clarity of the annotation guideline, and reduces Target 6 to a low-confidence version of Target 4.

The solution. Method: separate retrieval from adjudication completely. The three automated questions only retrieve candidates (Section 6.1); the decision comes from human policy reading against a definition of ambiguity that makes no reference to the model (Section 6.2). Evidence: check whether the distinction actually holds in the data. Of the 14 confirmed ambiguity findings, 10 came from the "model strongly disagrees" pool, and the two strongest involve no model hesitation at all — a distribution that could not occur if the two properties were the same thing. Control: two false positives (H0935, T0470) were kept on purpose to record explicitly that a probability near 0.5 is not sufficient to constitute a finding.

Difficulty 3: Preventing Neighbour Evidence from Contaminating Itself

The problem. The third signal for Target 4 is neighbour label agreement. But this dataset contains 415 duplicate rows. Without a restriction on the neighbour window, a message retrieves its own exact copy as its nearest neighbour and thereby "confirms" its own label. That would destroy the independence between the third signal and the first two — and independence is exactly what the protocol threshold requires.

The solution. The neighbour window strictly excludes every neighbour whose normalised text matches the target row, and the retrieval depth was raised from 9 to 20 so that a sufficient effective window remains after exclusion. There is an explicit trade-off here: for some rows the exclusion leaves too few valid neighbours, the third signal becomes unavailable, and those rows can reach at most two signals. We accept that loss of recall, because independence is a precondition of the protocol threshold and cannot be traded away.

Difficulty 4: Avoiding Evidence Retrieved by Model Signals Being Used to Prove the Model Better

The problem. Adjudication-aware evaluation (Section 7.2) removes held-out rows the audit found questionable. But Target 4 candidates are retrieved precisely because model signals disagree with their labels, and on held-out data "the model disagrees with this label" and "the model got it wrong" are the same event. Reporting the resulting score improvement would use the model's own errors to argue that the model is actually stronger — a complete circular argument.

The solution. Split the intervention into two levels and report them separately (ambiguous rows, then suspected label errors), state explicitly in the report that the second level carries no independent evidential weight, and decline to rely on it in the conclusions. What we do rely on is the first level: those rows were selected by human policy reading, independently of model correctness, and the model answered one of the two correctly — a result that in turn validates the argument in Section 6.2. The same discipline governs H0935: having judged it not ambiguous, we do not use ambiguity as grounds to remove it from scoring, even though removing it would improve the reported number.

---

10. Conclusions and Recommendations

Conclusion 1: data quality changes what the evaluation means. The headline 99.10% is shaped by three effects at once — cross-split leakage (5.7% of held-out rows already seen by the model), shallow shortcuts (98.11% reachable without representing meaning), and annotation noise (3 of 10 held-out errors are data problems). The number is not a measure of true model capability; it is the result of several data defects pointing in opposite directions and partly cancelling out.

Conclusion 2: this benchmark is now noise-dominated. With the data as it stands the accuracy ceiling is 99.73%, and the remaining headroom above the current score (7 rows) is roughly twice the disputed region (3 rows). Improvements smaller than about 0.3 percentage points cannot be distinguished from re-adjudicating a handful of annotation decisions.

Conclusion 3: the choice of metric is itself a data-quality issue. The nine interventions are nearly indistinguishable on accuracy but span 4.95 points of spam recall through unrelated mechanisms. Under a 13.4% class imbalance, reporting accuracy alone systematically hides the differences between data defects.

Conclusion 4: judgement matters more than flagging. Every target separates strong findings, uncertain findings, and false positives, and all adjudications are recorded with reasons in `adjudication_memo.csv` (444 entries, of which 158 are `false_positive_audit_finding`). Of the 670 submitted findings, only 303 make a strong claim, and each rests on mechanically checkable evidence. The value of a finding set lies in whether its confidence distribution can be explained, not in its length.

Recommendations:

1. **At the split level** — cross-split duplicates should be removed by cluster before evaluation, or the train/held-out split should be rebuilt by duplicate cluster so that all variants of one template fall on the same side.
2. **At the annotation level** — the guideline should explicitly define several recurring boundaries: commercial messages to users who have subscribed, charity appeals, the UK 070 number range, and personal messages that discuss or quote spam. These four account for most of the policy disputes in this audit.

3. **At the reporting level** — performance on this dataset should always be reported with spam recall and spam F1 alongside accuracy, together with a leakage-aware sensitivity result, rather than as a single accuracy figure.

---

11. AI Usage Declaration

AI tools were used in this project to draft candidate-retrieval scripts, generate evidence tables, and prepare initial policy readings. Every final audit decision was reviewed row by row by team members, who are responsible for the conclusions in this report.

Specifically: AI assisted with writing the signal scripts, generating candidate evidence tables, and drafting first-pass policy readings. Team members read the original messages, made the final accept/reject/relabel decisions, and verified that every number in this report matches the script outputs. AI-produced judgements were checked against the source text before being accepted. Rejected candidates and their reasons are recorded in `adjudication_memo.csv`.

No inter-annotator agreement statistic, such as Cohen's kappa, is claimed anywhere in this report, and no double annotation was simulated. Where uncertainty remains, it is recorded as a confidence value and as a rejected candidate rather than resolved silently. The dataset is the public UCI SMS Spam Collection. Dependency versions are listed in `requirements.txt` and `README.md`.

Appendix: Reproduction Commands

Environment: Python 3.12.11, scikit-learn 1.8.0, matplotlib 3.10.8, pandas 2.2.3, numpy 2.2.6, scipy 1.16.0. scikit-learn determines the numeric results; reproduction has been verified bit-for-bit (word held-out accuracy 99.1023%).

```
# Shared foundation
python scripts\download_data.py
python scripts\build_dataset.py
python scripts\train_baseline.py

# Dataset profile
python scripts\profile_data.py

# Targets 1-3: duplicates and leakage (order matters, see note below)
python scripts\audit_duplicates.py
python scripts\review_near_duplicates.py
python scripts\audit_leakage.py

# Targets 4 and 6: label errors and ambiguity
python scripts\audit_label_noise.py
python scripts\build_label_noise_outputs.py

# Target 5: shortcut features, including the promotional-token masking intervention
python scripts\audit_shortcuts.py

# Single-target intervention scripts
python scripts\label_overlay_experiment.py
python scripts\intervention_leakage_eval.py
python scripts\intervention_ambiguity_eval.py

# Unified intervention batch (source of the Section 7.1 table and the Section 7.5 cross-check)
python scripts\run_impact_interventions.py

# Build the two submission files (Section 8)
python scripts\build_suspicious_examples.py
python scripts\build_submission_outputs.py
```

The six audit reports are written to `audit/` (`profile.md`, `duplicates.md`, `leakage.md`, `label_noise.md`, `shortcut_features.md`, `ambiguity.md`), the intervention findings to `impact_analysis.md`, and the final step produces `suspicious_examples.csv` and `adjudication_memo.csv` while asserting the four protocol checks.

> **Note on ordering.** The three Target 1–3 scripts depend on each other: `audit_duplicates.py` rewrites `audit/duplicates.md` and generates the candidate queue, `review_near_duplicates.py` merges in the human decisions and appends the review section, and `audit_leakage.py` consumes the reviewed queue to compile cross-split evidence. The intervention scripts depend on the outputs of their respective targets and must run after them; the two build scripts must run after all audit scripts.